Critical-Span Retention and Query-Unknown Discovery for Long-Context KV Compression

Nils Persson Suorra

Preprint v1.0 doi:10.5281/zenodo.21894255

Abstract

Long-context inference is limited by KV-cache memory. Training-free eviction methods shrink the cache, but it is often unclear *which* tokens a task requires and whether online streaming fails from insufficient peak budget or weak **query-unknown discovery**. We study these questions on a controlled retrieval suite using Qwen3-4B on an RTX 3090, then test the resulting mechanism on a fixed public LongBench slice.

1. **Structure (H1′).** Within the measured 15-case protocol, retaining critical fact tokens plus a local radius $R^* = 1$ (with attention sinks and a recent/question window) is empirically **necessary and sufficient** for exact single-fact recall: oracle **15/15**, anti-oracle **0/15**, and full KV **15/15** (5 seeds $\times$ 3 depths @4k).
2. **Discovery gap.** At the tested peak class, attention stream@512 succeeds on **5/15**, while a perfect online pin of critical$\pm R$ succeeds on **15/15**. This intervention isolates discovery, rather than cache capacity alone, as the binding factor in these cases.
3. **Closing the gap (suite).** A **surface novelty detector** (within-prefix rarity plus digit/ID-like cues; no final question) reaches **14/15** @4k. With **sticky pin packing**, it reaches **9/9** at each measured length from 16k through **40k** at stream@512, with measured peak cache $\sim$**1k tokens**.
4. **Public long-doc QA (decomposition).** On a fixed 60-item LongBench slice, sticky novelty lags full ($0.64\times$ mean F1 at peak $\sim$1k). A **posthoc query-aware diagnostic** (full prefill $\rightarrow$ compress) reaches $0.95\times$ full mean F1 at final $B = 512$ and $\sim 1.01\times$ at $B = 1024$. **Query-hold** streaming trades peak for quality; its best measured point reaches $0.92\times$ full mean F1 at peak $\sim$2.5k.

The critical-span mechanism transfers across Qwen2.5, Llama-3.2, and hybrid Gemma-4 (full layers). We do **not** claim general long-context SOTA; see Section 6.

1 Introduction

KV cache memory grows with context length L. On a fixed workstation (here: 24 GB), full-cache decode becomes the bottleneck long before "the model cannot attend." Training-free compressors reduce tokens kept, but practice often optimizes average scores rather than **guaranteeing** the spans that encode answers. Online pipelines must also score tokens **before** the user question exists.

We study **near-lossless** compression on retrieval-critical prompts: quality Q should match full KV at $\varepsilon \approx 0$ under a stated multi-seed protocol. The systems objective is

$$L_\varepsilon \;=\; \max\big\{\, L : Q(M, L) \geq (1 - \varepsilon)\, Q(\text{Full}, L) \,\big\} \tag{1}$$

under **peak** cache / VRAM constraints, not only final decode size after a full prefill.

Claim. Our experiments support two separable questions: whether a retained set contains the task-critical local neighborhood, and whether an online policy can discover that neighborhood before the question arrives. On the measured open-QA slice, the resulting tradeoff is an **online retention / peak-cache Pareto**; we do not establish that this decomposition is universal.

2 Related work

Training-free KV eviction and selection. A large line of work compresses the KV cache without fine-tuning by dropping low-scoring tokens. **H2O** [1] keeps heavy hitters under accumulated attention; **StreamingLLM** [2] retains attention sinks plus a recent window; **Scissorhands** [3] and **TOVA** [4] prune by persistence, recency, or attention proxies; **SnapKV** [5] and **PyramidKV** [6] use observation-window attention and structured budgets. **Quest** [9] performs query-aware page selection, while **Ada-KV** [10] allocates budgets adaptively across heads. Our online setting is stricter in one respect: the final question is unavailable when early evictions occur. The contribution here is therefore diagnostic rather than a leaderboard claim: kill tests for a task-defined keep set, an oracle-online intervention that isolates discovery, and a simple surface-based policy evaluated under that intervention.

Head structure and external memory. **DuoAttention** [11] separates retrieval heads from streaming heads, and **RazorAttention** [12] similarly exploits retrieval-head structure. These head-wise approaches are orthogonal to our token-level packing rule and suggest a natural route to reduce the number of layers requiring full retrieval capacity. **InfLLM** [13] instead retrieves from an external memory. Such systems can relax eviction irreversibility; our experiments deliberately study a single-GPU retained-cache setting.

Streaming and long-context systems. Chunked prefill and stream compression cap **peak** memory rather than only final decode size. We report peak cache tokens, peak VRAM, and prefill latency on a consumer GPU. Hybrid architectures with sliding-window and full-attention layers still leave full-layer KV as compressible state; we compress full layers only and keep bookkeeping for hybrid score passes.

Quantization. KV quantization (e.g. **KIVI** [7]) reduces bytes per retained token and is complementary to eviction. We use fake-int8 only for logical byte accounting in some benches; quality results use bf16 caches unless noted.

Benchmarks. Needle-in-a-haystack and multi-hop synthetic suites stress retrieval under controlled depth and seed, but single-needle success can overstate broader long-context ability. **LongBench** [8], **RULER** [14], and **InfinityBench** [15] cover broader long-document, multi-document, aggregation, and very-long-context settings. We use a **fixed 60-item LongBench slice**, not a full leaderboard, to separate controlled-suite success from open QA and to measure posthoc and online peak-quality tradeoffs.

Positioning. We are not proposing a new attention kernel or claiming SnapKV-beating LongBench SOTA. The paper's contribution is a **mechanism + discovery diagnosis + workstation L_ε operating point**, with honest public-data limits and a quantitative online Pareto.

3 Methods

Task (suite). Needle-in-haystack style secrets in seeded filler; depths $\{0, 0.5, 1\}$; success = exact key recall (greedy decode). Stresses: NL facts, multi-needle, multi-hop, prose soft facts, multidoc. Multi-seed protocols: 5×3 at 4k; 3×3 at long L.

Evaluation design. Each arm uses the same prompt cells within a protocol. Success counts are descriptive proportions, not independent hypothesis tests: seeds change filler and secret values, while the three depths share a seed. For scale, Wilson 95% intervals are $[0.80, 1.00]$ for 15/15, $[0, 0.20]$ for 0/15, $[0.15, 0.58]$ for 5/15, $[0.70, 0.99]$ for 14/15, and $[0.70, 1.00]$ for 9/9. We report the intervals to show sampling uncertainty, but do not interpret clustered cells as 15 fully independent trials.

H1′ keep set.

$$K^\star \;=\; \text{sinks} \;\cup\; \text{critical} \pm R \;\cup\; \text{recent/question}.$$

Kill arms: full, oracle_r1 ($K^\star$), anti_oracle (drop critical$\pm R$). $R^* = 1$ is the measured default radius.

Posthoc scorer (`seed_valley`). After full prefill of length T, score the last observation window of size W (default $W{=}128$, covering the question when present) with shared attention votes over the prefix. Select seed peaks, grow contiguous valleys while score $\geq 0.25\times$ seed, expand $\pm R$, force-keep sinks and the last W tokens, pack to budget B. Used for suite tax tables and as the LongBench **posthoc upper bound** (full prefill $\rightarrow$ compress@B; peak cache $= T$).

Streaming. Chunked prefill (chunk size 512); whenever cache exceeds the active budget, compress online. We define *peak cache* as the maximum number of cached token positions observed before or after a compression event, so peak cache is approximately budget plus one chunk. Peak VRAM is CUDA's maximum allocated memory during the run. Absolute RoPE indices continue from original T at decode time after compress.

Oracle-online upper bound. Force-keep critical$\pm R$ whenever still in cache (perfect discovery) at the same peak class as stream@512.

Surface novelty (query-unknown). For token type t_i in a prefix of length n, let $c(t_i)$ be its prefix count and $\mathbb{1}_{\text{first}}(i)$ mark its first occurrence. The implemented raw score is

$$s_i = \log\left(1 + \frac{n}{c(t_i)}\right) + 0.75\mathbb{1}_{\text{first}}(i) + 2.5d_i + 2.0u_i + 1.0m_i + 0.8p_i,$$

where d_i marks a digit, u_i an ID-like token piece, m_i mixed case or alphanumeric form, and p_i is its non-alphanumeric character fraction. A width-3 local max filter is applied; positions above a fixed fraction of the prefix maximum are joined into components and expanded by $R = 1$. At most 12 highest-scoring components are proposed. A **sticky** registry retains discovered pins until packing must drop them; packing reserves sinks and recent tokens, then ranks pins by novelty score. This deterministic heuristic sees no final question and is not a learned importance head.

Streaming packing procedure. For each 512-token chunk: (1) append the chunk with its absolute positions; (2) update novelty scores on the observed prefix; (3) merge new components into the sticky registry; and (4) if the active cache exceeds the budget, retain sinks, recent tokens, and the highest-ranked registered pins up to budget. After the final prompt chunk, compress to the final budget and recompute the first-token logits under that compressed cache. The implementation in `experiments/novelty_detect.py` is the executable specification.

Query-hold (systems lever). Mid-stream: hybrid pins (novelty $\cup$ attention peaks) under the hold budget. Near end / final pass: query-aware re-rank and tighten to the final budget once the question is in the observation window. Peak $\approx$ hold + chunk.

Decode after compress. Greedy decode uses absolute `next_position`$= T$. After any final compress we **recompute first-token logits** under the compressed cache (re-forward the last prompt token) so the first generated token is not scored from a stale full/hold cache.

Public LongBench slice. THUDM/LongBench: first 20 items each of `multifieldqa_en`, `qasper`, `hotpotqa` (60 total); contexts truncated to max 4096 tokens (head+tail); token F1 vs gold + substring hit; greedy, max_new=64; same items for all arms. Absolute full F1 is modest under 4B/greedy/truncate, so we report **ratios to same-item full**.

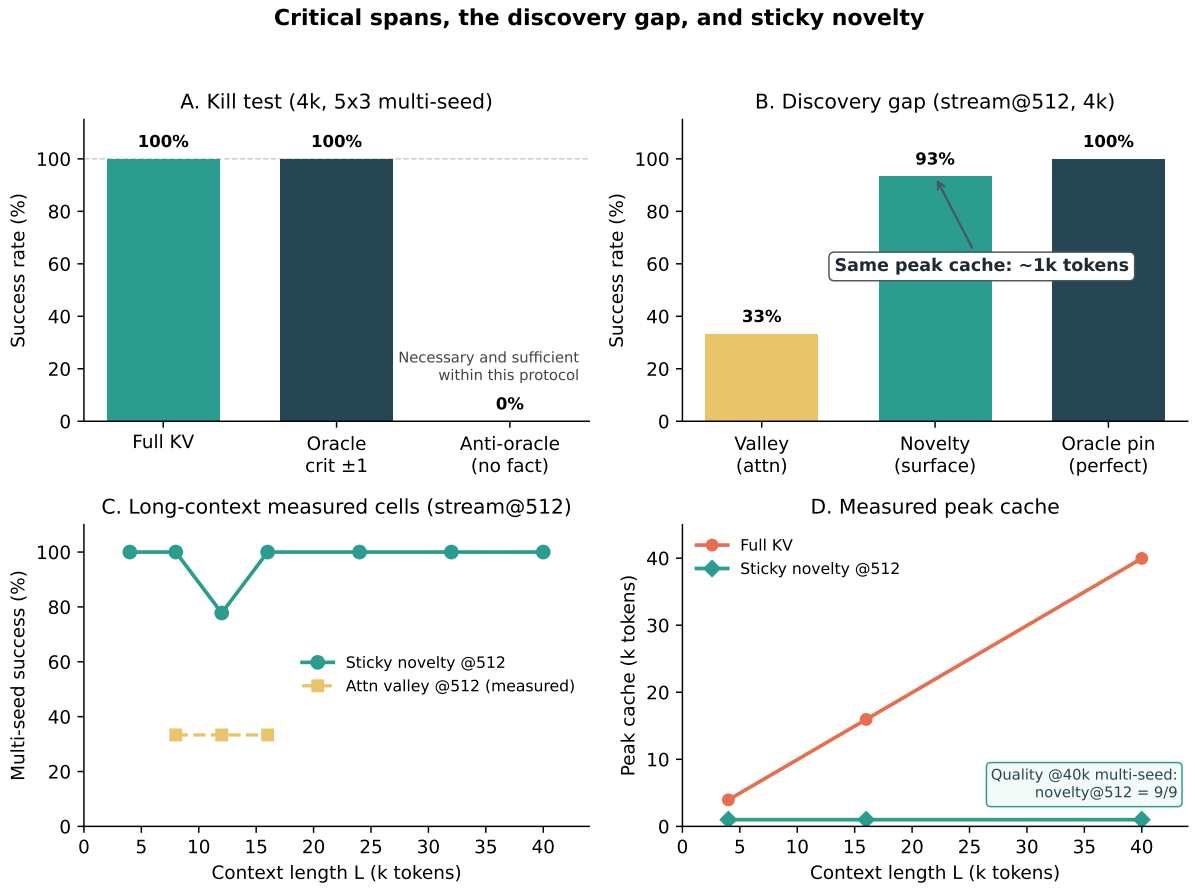

Figure 1: **Story figure.** **(A)** H1′ kill test @4k multi-seed (5×3): full and oracle crit± 1 both 15/15; anti-oracle 0/15. **(B)** Discovery gap at stream@512: valley 33%, surface novelty 93%, oracle pin 100%. **(C)** Long-L multi-seed success: sticky novelty@512 reaches 9/9 at the measured 16k, 24k, 32k, and 40k cells; attention valley@512 was measured only through 16k. **(D)** Measured peak cache is $\sim$1k under sticky novelty stream@512 at 4k, 16k, and 40k, while full KV grows with L.

Suite-alignment risk. Surface novelty is favored when secrets are surface-distinct against repetitive filler. We report prose/NL/adversarial stresses and public LongBench as honesty checks; LongBench is where novelty *fails* to match full at peak$\sim$1k.

Default API. `prefill_auto(..., mode="stream", discovery="novelty")`. Use `discovery="query_hold"` for the peak/quality Pareto path.

4 Results

4.1 Figure 1: Story in four panels

A. H1′ kill (4k, 5×3 multi-seed). Full and oracle crit± 1 both **15/15**; anti-oracle **0/15**. Bare critical tokens without radius often corrupt trailing digits (H1 needs local context); $R^* = 1$ restores $\varepsilon \approx 0$. Mean $|\text{oracle}| \approx$ **149** tokens vs full $\sim$4k.

B. Discovery gap (stream@512). Attention valley reaches **5/15**; surface novelty reaches **14/15**; and oracle pin reaches **15/15**. Their peak budgets are in the same class ($\sim$1k with a chunk). Because the oracle changes token selection while holding the budget class fixed, these cells identify the detector as the binding factor for valley@512 in this protocol.

C. Long L multi-seed (3×3). Sticky novelty@512 is **9/9** at 16k, 24k, 32k, and **40k**. Non-sticky novelty degraded at 24k ($\sim$6/9) during method development; sticky packing reached 9/9. Valley@512 was 3/9 at 8k, 12k, and 16k. We did not run that baseline at 24k, 32k, or 40k, and Figure 1 does not extrapolate it.

D. Peak cache. In the resource runs at 4k, 16k, and 40k, full KV scales with L, while sticky novelty stream@512 stays at $\sim$**1k** peak cache tokens. These resource rows are single seed-0 operating-point measurements, not replicated latency benchmarks.

4.2 Scorer tax (posthoc, question known)

Table 1: Posthoc budget tax vs oracle keep set (4k multi-seed).

Method	All-cell $B_{\min}$	Tax vs mean $\lvert\text{oracle}\rvert$
oracle_r1	$\sim$149	$1.0\times$
seed_valley (multi-seed all-cell)	**192**	$\sim$**1.29**$\times$ / mean cell tax $\sim$**1.16**$\times$
SnapKV (single-needle mid, same scorer suite)	192	$\sim 1.24\times$ vs oracle mid
seed_valley (single-needle mid)	**176**	$\sim$**1.14**$\times$ vs oracle mid

Posthoc is near-oracle-tight on the multi-seed suite; SnapKV is competitive but seed_valley is slightly tighter at mid-depth single-needle (FINDINGS scorer budget). **Stream is not** near-oracle until discovery improves.

4.3 Stress and transfer (summary)

Table 2: Stress suite and cross-model transfer (summary).

Check	Result
NL facts (names/places) sticky novelty@512	**15/15** multi-seed
Code needles sticky	**15/15**
Adversarial ID-like filler (pre-sticky)	novelty **100%** vs valley **33%**
Multi-needle recall_all @512	novelty **5/5** (`max_new`$\geq$96); valley **0/5**
2-hop (Alice$\rightarrow$id$\rightarrow$password) @512	novelty **5/5**; valley **1/5** (needs @1024 for 5/5)
3-hop + distractors @512	novelty **5/5**; valley **0/5** (needs @1024 for 5/5)
Prose soft fact (no digits) @512	novelty **15/15**; valley 53%
Multidoc (6 titled docs) @512	novelty **15/15**; valley/oracle_pin **5/15**
External-style mixed QA ($10\times\sim$4k)	novelty **10/10** hits (= full); valley **0/10**
Gemma-4 E4B hybrid novelty@512	multi-seed **9/9** @4k (valley needs $\sim$1024)
Qwen2.5 / Llama-3.2	H1 holds; family-specific posthoc floors

4.4 Public LongBench decomposition

Synthetic suite success does not automatically transfer to open long-document QA. On the fixed 60-item slice (same items for all arms), online novelty stream@512 is far below full ($0.64\times$ mean F1 at peak $\sim$1k; Table 4). A posthoc query-aware diagnostic uses full prefill, then `seed_valley` compression with the question in the observation window. It reaches **0.95**$\times$ full mean F1 at final B=512 and $\sim$**1.01**$\times$ at 1024/2048 (Table 3). Peak remains full L, so this is not an online memory result. It shows only that this scorer can preserve same-item aggregate quality in this setting when both the full prefix and question are available.

Table 3: LongBench slice (60 items @4k truncate): posthoc upper bound vs full. Peak for posthoc is full L (not a free systems win).

Arm	Mean F1	Substr hit	Peak	F1 / full
full	**0.282**	25%	$= L$	$1.00\times$
posthoc@512	0.267	27%	$= L$	$\mathbf{0.95}\times$
posthoc@1024	0.284	28%	$= L$	$\sim 1.01\times$
posthoc@2048	0.285	27%	$= L$	$\sim 1.01\times$

Online paths must drop tokens before the question exists. Table 4 reports a **query-hold Pareto**: larger mid-stream hold improves recovery after query-aware tighten; final budget 1024 consistently beats 512 at fixed hold. The best measured online point, hold $2048 \rightarrow$ final 1024, reaches $\approx \mathbf{0.92}\times$ full F1 at mean peak $\sim$2.5k tokens. This lies between novelty@1k ($0.64\times$) and full@L.

Table 4: LongBench slice: online quality vs peak cache (query-hold Pareto).

Arm	Mean F1	Hit	Mean peak	F1 / full
novelty@512	0.179	10%	1024	$0.64\times$
query_hold h1024→f512	0.199	18%	1536	$0.70\times$
query_hold h1024→f1024	0.231	22%	1536	$0.82\times$
query_hold h2048→f512	0.212	17%	$\sim$2531	$0.75\times$
query_hold h2048→f1024	**0.260**	22%	$\sim$**2531**	$\mathbf{0.92}\times$
query_hold h4096→f1024	0.249	28%	$\sim$3876	$0.88\times$

Interpretation. Open long-doc QA on this slice is not primarily "seed_valley cannot find answers when query-aware." It is an **online retention** problem. Systems claims should be stated as a peak/quality curve, not as free near-full quality at peak$\sim$1k outside retrieval suites.

4.5 Systems resources (peak VRAM / latency)

Mid-depth needle, seed 0. Novelty stream@512 keeps peak VRAM and decode KV essentially flat:

Table 5: Peak resources: full chunked prefill vs sticky novelty stream@512.

L	full VRAM	nov. VRAM	full KV	nov. KV	nov. prefill
4k	8.5 GB	**8.3 GB**	569 MB	**72 MB**	1.6 s
16k	10.6 GB	**8.3 GB**	2296 MB	**72 MB**	5.9 s
40k	14.5 GB	**8.3 GB**	5754 MB	**72 MB**	17.6 s

Peak cache under novelty is 1024 tokens at all three measured lengths. Resource and latency values are one run per length and should be read as implementation operating points.

5 Adaptive policy (systems)

`prefill_auto` applies L-based budgets + per-model floors. With default `discovery="novelty"`, single-needle stream@**512** is the measured multi-seed operating point through **40k**. Attn-only discovery still needs larger floors ($\sim$1536 multi-seed @4k). Hybrid Gemma: compress **full** layers only; sliding layers keep absolute prefix bookkeeping for score-pass.

6 Limitations and what we do *not* claim

6.1 What we claim

- In the **15 measured cells** of the controlled 4k single-fact protocol, critical$\pm R^*$ is empirically necessary and sufficient for exact recall.
- At stream@512 on those cells, an oracle-online intervention raises success from 5/15 for attention valley to 15/15, isolating query-unknown discovery as the binding factor for that comparison.
- Sticky surface novelty reaches **9/9** at every measured long-context cell from 16k through **40k**, with peak cache $\sim$1k on the primary model. Transfer results are smoke tests on small instruct models, not broad model-family validation.
- On a fixed public LongBench slice, **posthoc query-aware compression reaches 0.95$\times$ full F1** at final $B{=}512$ (peak=L), while online novelty lags ($\sim$0.64$\times$ at peak$\sim$1k); **query_hold** recovers a measured Pareto (best $\sim$0.92$\times$ full F1 at peak $\sim$2.5k). Absolute full F1 is modest under 4B/greedy/4k truncate.

Table 6: Scope boundaries (honest).

Not claimed	**Why**
General long-context SOTA	No full RULER / LongBench / InfiniteBench leaderboard evaluation.
All task types	Primary suite is retrieval / needle-class (+ multi-needle, hops, prose, multidoc stress). Full reasoning chains, code repos, and full LongBench/RULER leaderboards untested.
Oracle-tight online budgets for free	Sticky novelty multi3/hop is **5/5**@512 with adequate decode budget; multi-secret packing still stresses oracle_pin@512 (multi3 **2/5**). Suite-alignment of surface novelty remains.
Production memory stack	Fake-int8 is logical accounting only; no fused kernels or serving productization.
Novelty as universal "importance"	Exploits surface distinctness vs repetitive filler; ID-like filler / filler-like secrets can still confuse discovery.
Large models / training-free SOTA	Primary evidence is $\sim$3–4B instruct models on one GPU class.
Statistical finality	Multi-seed N modest and depth cells share seeds. Wilson intervals are wide; this is a controlled study, not a large meta-analysis.
$\varepsilon = 0$ beyond measured envelope	Sticky multi-seed measured through **40k**; longer L is extrapolation.
Peak VRAM = final decode KV	Streaming caps **peak cache tokens**; weights and activations still dominate total VRAM.

6.2 Threats to validity

- **Greedy decode** and chat templates may interact with exact-match / token-F1 scoring; LongBench absolute full F1 is low ($\sim$0.28).
- **Filler distribution** is synthetic; surface novelty is suite-aligned to distinct secrets in repetitive hay. LongBench is the external honesty check.
- **Multidoc packing:** novelty@512 can beat labeled oracle_pin@512 (15/15 vs 5/15), so keep-set definitions and packing under multi-span budgets remain open.
- **Hybrid models:** only full layers are compressed; sliding-window behavior is infrastructure-sensitive.
- **Sample size:** multi-seed N is lab-grade (5×3 @4k; 3×3 long-L); "9/9 through 40k" is cells on this protocol, not a large meta-analysis.

- **Unreplicated systems rows:** peak VRAM and latency are single seed-0 runs per length; no variance or cross-hardware measurements are reported.
- **Environment capture:** the repository records dependency ranges, but the exact historical package lockfile for the reported runs was not preserved.
- **Selection bias toward positive arms:** we iterated methods until discovery worked; negative results (scored capsules alone, non-sticky long-L) are reported but the path is research-in-the-loop.
- **Community baselines:** stream tables emphasize valley / novelty / oracle / query_hold; SnapKV is reported posthoc on the scorer-tax suite, not as a full multi-seed stream LongBench arm.

7 Conclusion

The controlled experiments support a useful decomposition of training-free KV compression: retain the task-critical local neighborhood, and discover it before irreversible eviction. Within the tested retrieval protocol, critical± 1 matches full recall and its removal eliminates recall; an oracle-online intervention then isolates discovery at a fixed peak class. Sticky surface novelty reaches 9/9 at each measured length from 16k through 40k with a measured $\sim$1k-token peak cache. The fixed LongBench slice is the important counterweight: novelty reaches only $0.64\times$ full mean F1 at that peak, while query-hold reaches a measured $0.92\times$ at $\sim$2.5k. The evidence therefore supports a protocol-specific mechanism and an online peak-quality tradeoff, not general near-lossless long-context compression. Stronger community baselines, broader public benchmarks, and replicated systems measurements are the next steps.

Reproducibility

```
# Multi-seed H1 + scorer tax
python experiments/bench_paper_rigor.py --seeds 0,1,2,3,4 --ctx 4096

# Discovery gap / novelty
python experiments/bench_capsules.py --novelty --oracle-online \
  --skip-capsules --budgets 512
python experiments/bench_novelty_stress.py
python experiments/bench_novelty_longL.py \
  --ctx 16384,24576,32768,40960 --arms novelty --budgets 512

# LongBench posthoc UB + query_hold Pareto (60 items)
python experiments/bench_external_slice.py --source longbench --n 60 \
  --arms full,posthoc --budgets 512,1024,2048
python experiments/bench_external_slice.py --source longbench --n 60 \
  --arms full,novelty,query_hold --budgets 512 \
  --hold-budgets 1024,2048,4096 --final-budgets 512,1024

# Figure + transfer
python experiments/plot_paper_figures.py
python experiments/bench_transfer.py --model google/gemma-4-E4B-it
```

Primary model: `Qwen/Qwen3-4B-Instruct-2507`, transformers, CUDA bf16, RTX 3090.
Artifact index: `results/FINDINGS.md`.
Machine-readable aggregates: `results/paper_aggregates.csv`; source-run mapping and protocol notes are in `results/PROVENANCE.md`.
Markdown twin: `papers/PAPER_DRAFT.md`.
Build PDF: from `papers/`, run `powershell -File build_pdf.ps1`.

References

[1] Z. Zhang et al. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS*, 2023.

[2] G. Xiao et al. Efficient Streaming Language Models with Attention Sinks. *ICLR*, 2024.

[3] Z. Liu et al. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. *NeurIPS*, 2023.

[4] M. Oren et al. Transformers are Multi-State RNNs (TOVA). *arXiv:2401.06104*, 2024.

[5] Y. Li et al. SnapKV: LLM Knows What You are Looking for Before Generation. *NeurIPS*, 2024.

[6] Z. Cai et al. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *arXiv:2406.02069*, 2024.

[7] Z. Liu et al. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *ICML*, 2024.

[8] Y. Bai et al. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *ACL*, 2024.

[9] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. *arXiv:2406.10774*, 2024.

[10] Y. Feng, J. Lv, Y. Cao, X. Xie, and S. K. Zhou. Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. *arXiv:2407.11550*, 2024.

[11] G. Xiao, J. Tang, J. Zuo, J. Guo, S. Yang, H. Tang, Y. Fu, and S. Han. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. *arXiv:2410.10819*, 2024.

[12] H. Tang et al. RazorAttention: Efficient KV Cache Compression Through Retrieval Heads. *arXiv:2407.15891*, 2024.

[13] C. Xiao, P. Zhang, X. Han, G. Xiao, Y. Lin, Z. Zhang, Z. Liu, and M. Sun. InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. *arXiv:2402.04617*, 2024.

[14] C.-P. Hsieh et al. RULER: What's the Real Context Size of Your Long-Context Language Models? *COLM*, 2024.

[15] X. Zhang et al. InfinityBench: Extending Long Context Evaluation Beyond 100K Tokens. *ACL*, 2024.